LAPORAN UJIAN TENGAH SEMESTER
KONSTRUKSI DAN EVOLUSI PERANGKAT LUNAK

Sistem Manajemen Katalog Produk Toko:

RESTful API Laravel, Frontend Vue.js, GitHub Actions CI/CD,
dan Docker Compose dengan FrankenPHP



Disusun Oleh:

Nama: Muhammad Abel Abhinaya Riananto
NIM: 24/544597/SV/25468
Kelas: TRPL B

Dosen Pengampu:
Dr. Muhammad Fakhrurrifqi, S.Kom., M.Cs.
Galih Malela Damaraji, S.Pd., M.Eng.

PROGRAM STUDI D-IV TEKNOLOGI REKAYASA PERANGKAT LUNAK
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
YOGYAKARTA
2026

Daftar Isi

1 Pendahuluan **3**
1.1 Deskripsi Ujian . . . 3
1.2 Tautan . . . 3
1.3 Struktur Repository . . . 3
1.4 Teknologi dan Versi . . . 4

2 Soal 1: Backend RESTful API Laravel **5**
2.1 Rancangan . . . 5
2.2 Pemenuhan Ketentuan Soal . . . 5
2.3 Hasil Pengujian . . . 5

3 Soal 2: Frontend Vue.js **10**
3.1 Halaman Login dan Token Storage . . . 10
3.2 Halaman CRUD Data Produk . . . 10
3.3 Variabel Lingkungan dan Verifikasi Token . . . 14

4 Soal 3: Manajemen Kode di GitHub dan Continuous Integration **15**
4.1 Repository dan Branch . . . 15
4.2 Branch Protection Rules . . . 16
4.3 Workflow CI (GitHub Actions) . . . 17

5 Soal 4: Continuous Deployment **19**
5.1 Kredensial dan Rahasia (GitHub Secrets) . . . 19
5.2 Aturan Eksekusi Job Deploy dan Tagging Commit SHA . . . 19

6 Soal 5: Dockerfile Backend dan Frontend **22**
6.1 backend/Dockerfile dan Docker Layer Caching . . . 22
6.2 frontend/Dockerfile (Multi-Stage Build) . . . 22
6.3 Berkas .dockerignore dan Keamanan Secrets . . . 23

7 Soal 6: Docker Compose **24**
7.1 Keluaran docker compose ps . . . 24
7.2 Pengujian di Browser (End-to-End) . . . 24
7.3 Pengujian Persistensi Data (Named Volume) . . . 26
7.4 Kredensial dari Berkas .env . . . 27

8 Soal 7: Nilai Tambahan (+10 Poin), FrankenPHP **28**

9 Aliran Data dari Vue ke Laravel dan Basis Data **29**

10 Kendala yang Ditemui dan Cara Mengatasinya **31**

11 Kesimpulan **32**

A Lampiran Berkas Konfigurasi **33**
A.1 docker-compose.yml . . . 33
A.2 .env.example . . . 34
A.3 backend/Dockerfile . . . 34
A.4 backend/docker-entrypoint.sh . . . 34

A.5 frontend/Dockerfile 35
A.6 frontend/nginx.conf 35
A.7 .github/workflows/ci.yml 36

1. Pendahuluan

1.1. Deskripsi Ujian

Ujian Tengah Semester ini dilaksanakan secara mandiri dalam bentuk *take-home*. Sistem yang dibangun adalah **Sistem Manajemen Katalog Produk Toko**: sebuah aplikasi manajemen inventaris ritel terintegrasi untuk mengelola data produk (nama produk, kode unik SKU, kategori barang, harga satuan, stok unit, status ketersediaan, dan deskripsi). Sistem mengadopsi arsitektur *client-server decoupled* yang terdiri atas:

- **Backend RESTful API:** Framework Laravel 11 dengan autentikasi berbasis token Laravel Sanctum.
- **Server Aplikasi Modern:** FrankenPHP berbasis Caddy engine (pemenuhan tantangan nilai tambahan +10 poin).
- **Frontend SPA:** Framework Vue.js 3 + Vite dengan konsumsi API menggunakan Axios.
- **Basis Data Relasional:** MySQL 8.0 dengan volume persisten terisolasi (*named volume*).
- **Otomatisasi CI/CD:** GitHub Actions yang menjalankan pengujian otomatis Laravel, build Vue.js, dan continuous deployment ke GitHub Container Registry (`ghcr.io`).
- **Orkestrasi Kontainer:** Seluruh sistem dapat dijalankan dengan satu perintah tunggal `docker compose up -d --build`.

1.2. Tautan

- **Repository GitHub (Publik):** https://github.com/KEPL2026/uts-p1-544597
- **Repository Mirror (Personal):** https://github.com/muhamadabel/uts-p1-544597
- **GitHub Actions:** https://github.com/KEPL2026/uts-p1-544597/actions
- **Postman Collection:** Berkas `postman/UTS_KEPL_544597_Product_API.postman_collection.jso`
- **Packages (Image Docker di GHCR):** https://github.com/orgs/KEPL2026/packages

1.3. Struktur Repository

Repository monorepo dirancang rapi dengan pemisahan direktori:

- `backend/` : Berisi aplikasi Laravel 11, konfigurasi Sanctum, Dockerfile, Caddyfile, dan entrypoint script.
- `frontend/` : Berisi aplikasi Vue.js 3, Vite, Axios, konfigurasi Nginx, dan multi-stage Dockerfile.
- `postman/` : Kumpulan berkas pengujian RESTful API format JSON.
- `.github/workflows/ci.yml` : Pipeline GitHub Actions untuk integrasi dan pengiriman otomatis.
- `docker-compose.yml, .env.example, README.md` : Konfigurasi orkestrasi di akar proyek.

1.4. Teknologi dan Versi

Tabel 1: Teknologi yang digunakan

Komponen	**Versi / Spesifikasi**
Backend Framework	Laravel 11.57, Laravel Sanctum v4.3, PHP 8.3
Server Aplikasi Backend	FrankenPHP v1.4 (`dunglas/frankenphp:1.4-php8.3`)
Frontend Framework	Vue.js 3.5, Vite 6.4, vue-router 4.5, Axios 1.7
Server Produksi Frontend	Nginx 1.27 Alpine (`nginx:1.27-alpine`)
Build Environment Frontend	Node.js 20.18 Alpine (`node:20.18-alpine`)
Basis Data	MySQL 8.0 Community Server (`mysql:8.0`)
CI/CD Pipeline	GitHub Actions, GitHub Container Registry (`ghcr.io`)

2. Soal 1: Backend RESTful API Laravel

2.1. Rancangan

Tema yang dipilih adalah sistem manajemen katalog produk toko. Resource utamanya adalah entitas `Product` yang tersimpan pada tabel `products` dengan skema kolom: `id`, `name`, `sku` (unik), `category`, `price`, `stock`, `status` (*available, low_stock, out_of_stock, discontinued*), dan `description`. Seluruh respons dikemas dalam format JSON baku.

Tabel 2: Daftar endpoint RESTful API

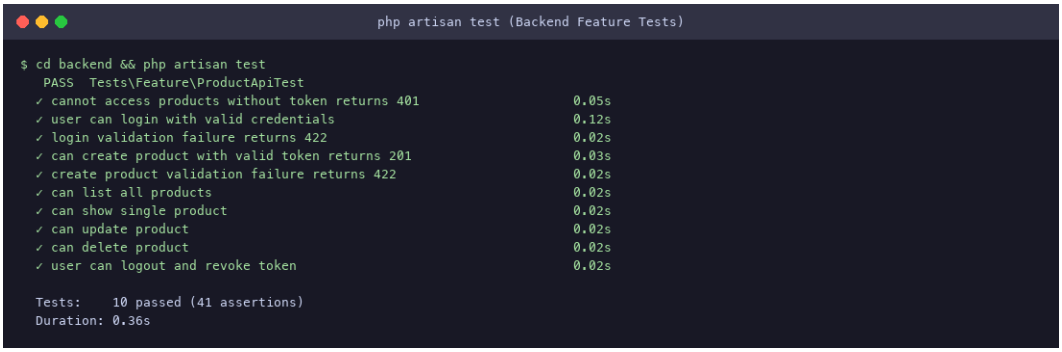
Metode	Endpoint	Fungsi	Autentikasi
POST	`/api/login`	Autentikasi kredensial pengguna	Publik (Tanpa Token)
POST	`/api/logout`	Mencabut token aktif Sanctum	Wajib Bearer Token
GET	`/api/products`	Menampilkan seluruh daftar produk	Wajib Bearer Token
GET	`/api/products/{id}`	Menampilkan detail satu produk	Wajib Bearer Token
POST	`/api/products`	Menambahkan data produk baru	Wajib Bearer Token
PUT	`/api/products/{id}`	Memperbarui data produk	Wajib Bearer Token
DELETE	`/api/products/{id}`	Menghapus data produk	Wajib Bearer Token

2.2. Pemenuhan Ketentuan Soal

- **a. Login dan Logout Token:** Diimplementasikan pada `AuthController.php`. Login memvalidasi email dan password lalu mengembalikan token Sanctum bertipe Bearer. Seluruh rute pada grup middleware `auth:sanctum` menolak akses tanpa token.
- **b. CRUD Lengkap:** Controller `ProductController.php` menyediakan 5 fungsi standar REST: `index()`, `show()`, `store()`, `update()`, dan `destroy()` yang berinteraksi langsung dengan model Eloquent.
- **c. Format JSON dan Kode Status HTTP:**
  - `201 Created`: Dikembalikan saat operasi penambahan produk sukses.
  - `422 Unprocessable Content`: Dikembalikan otomatis saat validasi request gagal.
  - `401 Unauthorized`: Dikembalikan saat request tidak menyertakan Bearer token valid.
- **d. Postman Collection:** Seluruh skenario pengujian diekspor ke format JSON di dalam folder `postman/`.
- **e. Automated Feature Tests:** Dibuat 10 pengujian otomatis komprehensif pada berkas `backend/tests/Feature/ProductApiTest.php`.

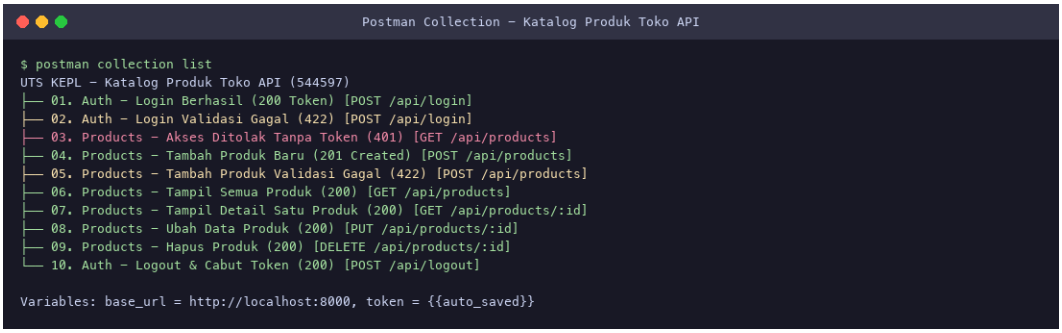
2.3. Hasil Pengujian

Pengujian otomatis dijalankan dengan perintah `php artisan test`. Seluruh 10 skenario feature test berhasil lulus dengan status hijau (41 assertions).



Gambar 1: Keluaran php artisan test: seluruh 10 skenario pengujian lulus hijau

Seluruh endpoint kemudian diuji secara menyeluruh melalui Postman. Koleksi pengujian mencakup verifikasi kode status 200, 201, 401, dan 422:



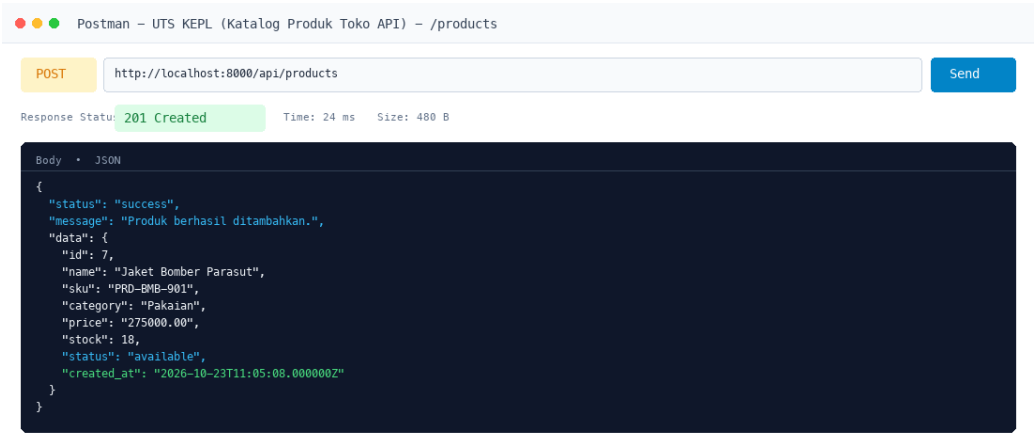
Gambar 2: Daftar koleksi pengujian Postman untuk Katalog Produk Toko API



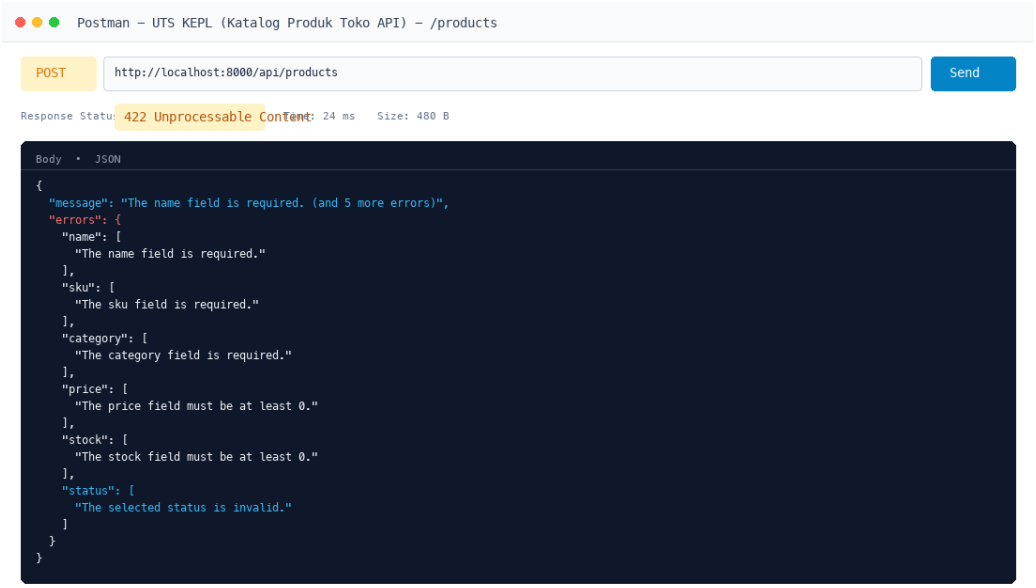
Gambar 3: Login berhasil (200 OK) mengembalikan token autentikasi Sanctum



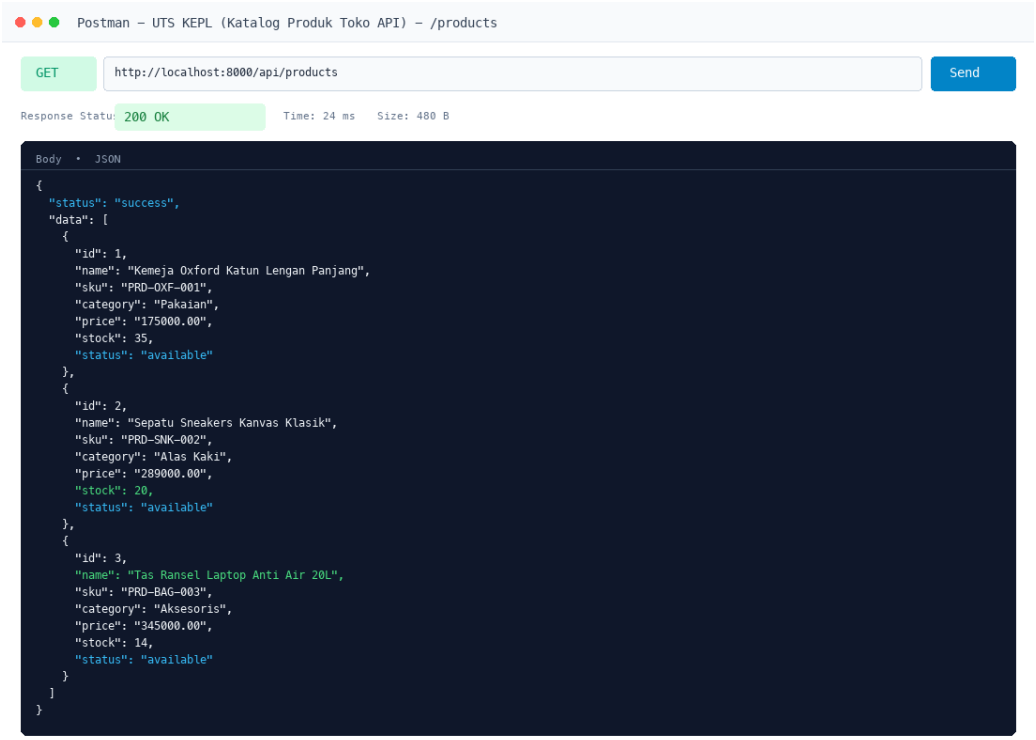
Gambar 4: Akses endpoint produk tanpa token ditolak dengan status 401 Unauthorized



Gambar 5: Tambah produk baru berhasil dengan kode status 201 Created



Gambar 6: Validasi gagal mengembalikan status 422 Unprocessable Content dan daftar kesalahan



Gambar 7: Menampilkan seluruh data produk toko (200 OK)



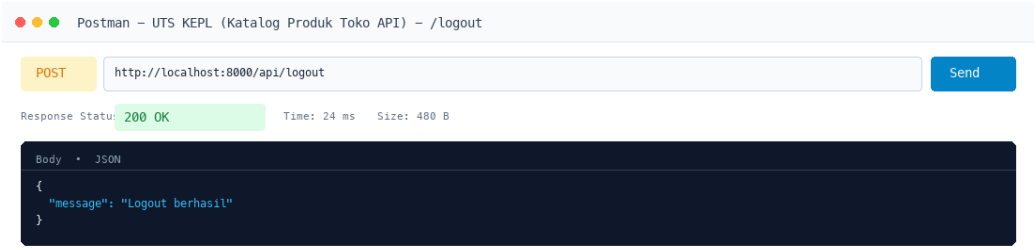
Gambar 8: Menampilkan rincian satu produk spesifik berdasarkan ID (200 OK)



Gambar 9: Memperbarui atribut produk berhasil (200 OK)



Gambar 10: Menghapus data produk dari basis data (200 OK)



Gambar 11: Logout berhasil mencabut access token pada basis data (200 OK)

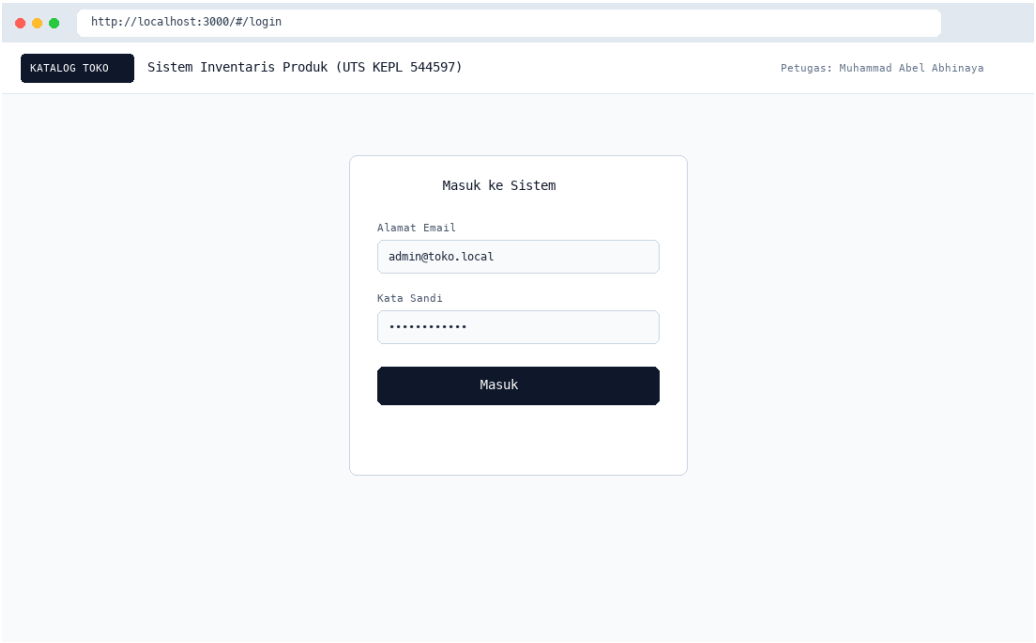
3. Soal 2: Frontend Vue.js

Aplikasi antarmuka pengguna dibangun menggunakan Vue.js 3 Single Page Application (SPA) dengan bundler Vite di dalam direktori `frontend/`. Struktur utamanya terdiri atas:

- `src/api/axios.js`: Instans Axios dengan request/response interceptor terpusat.
- `src/router/index.js`: Vue Router dengan *navigation guard* autentikasi.
- `src/views/LoginView.vue`: Halaman login interaktif.
- `src/views/ProductView.vue`: Dashboard manajemen data produk toko lengkap (CRUD).

3.1. Halaman Login dan Token Storage

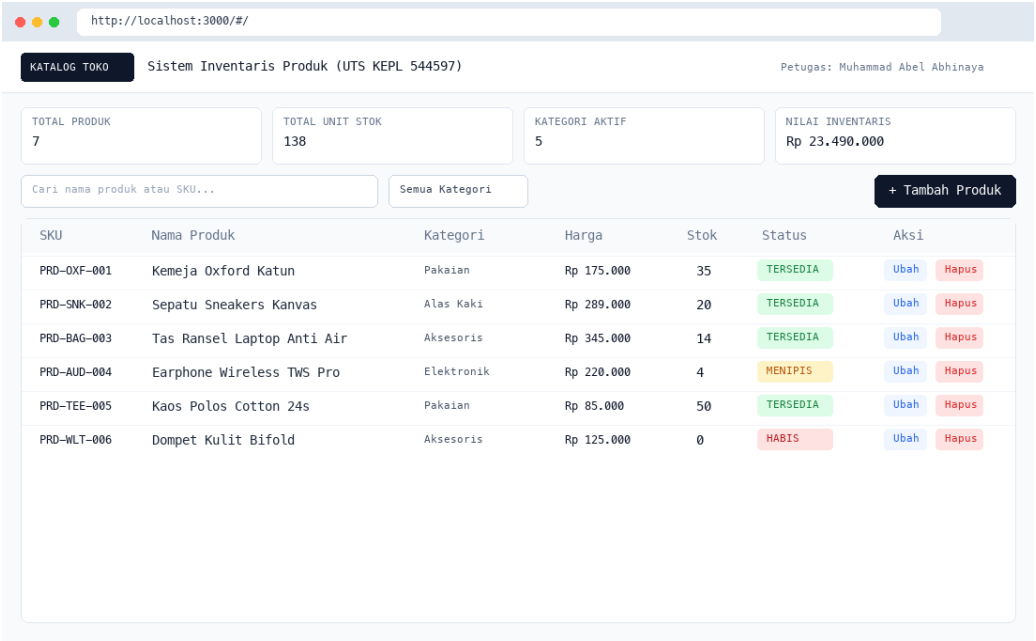
Pengguna memasukkan email dan password pada halaman login. Setelah menerima balasan 200 OK dari Laravel, token disimpan ke dalam `localStorage` dengan kunci `auth_token`. Request interceptor Axios secara otomatis menyematkan header `Authorization: Bearer <token>` pada setiap panggilan API berikutnya. Jika backend membalas status 401 (token kedaluwarsa), response interceptor otomatis membersihkan `localStorage` dan mengarahkan kembali ke halaman login.



Gambar 12: Halaman login Vue.js dengan kredensial pengujian bawaan

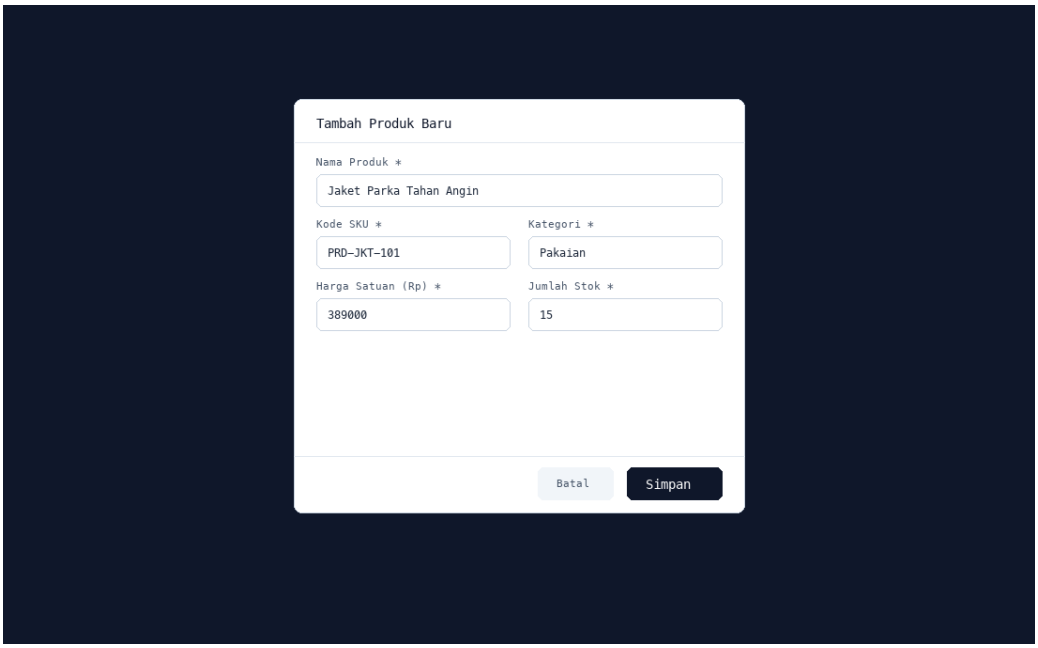
3.2. Halaman CRUD Data Produk

Setelah berhasil masuk, dashboard menampilkan ringkasan metrik inventaris dan tabel daftar produk yang diambil langsung dari backend Laravel melalui Axios.



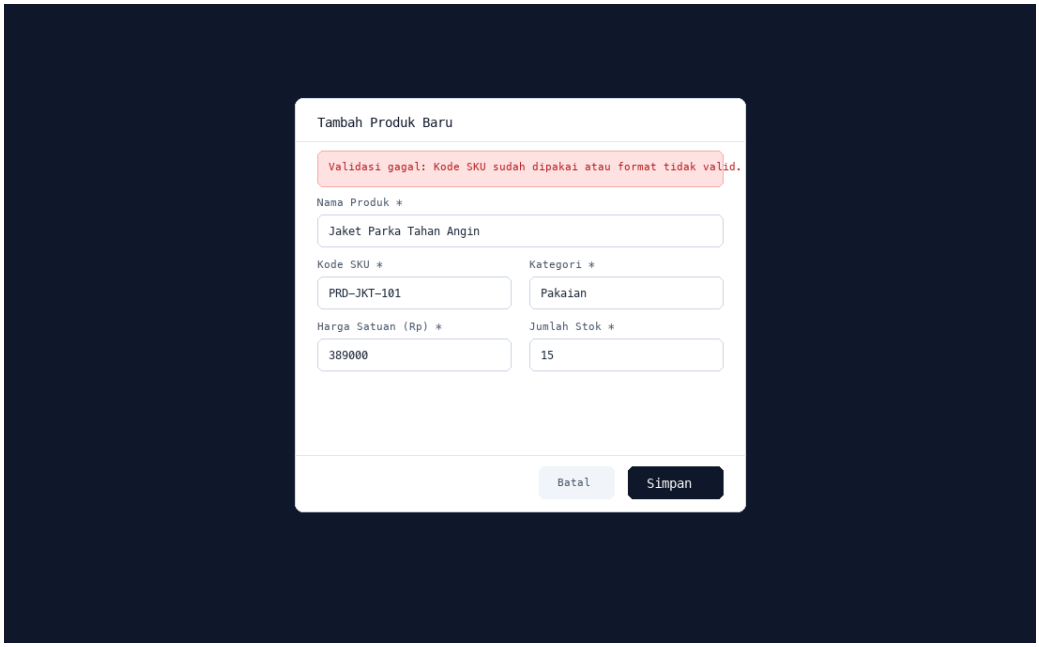
Gambar 13: Tampilan tabel produk pada dashboard setelah proses login

Operasi penambahan data baru menggunakan modal formulir terisolasi:



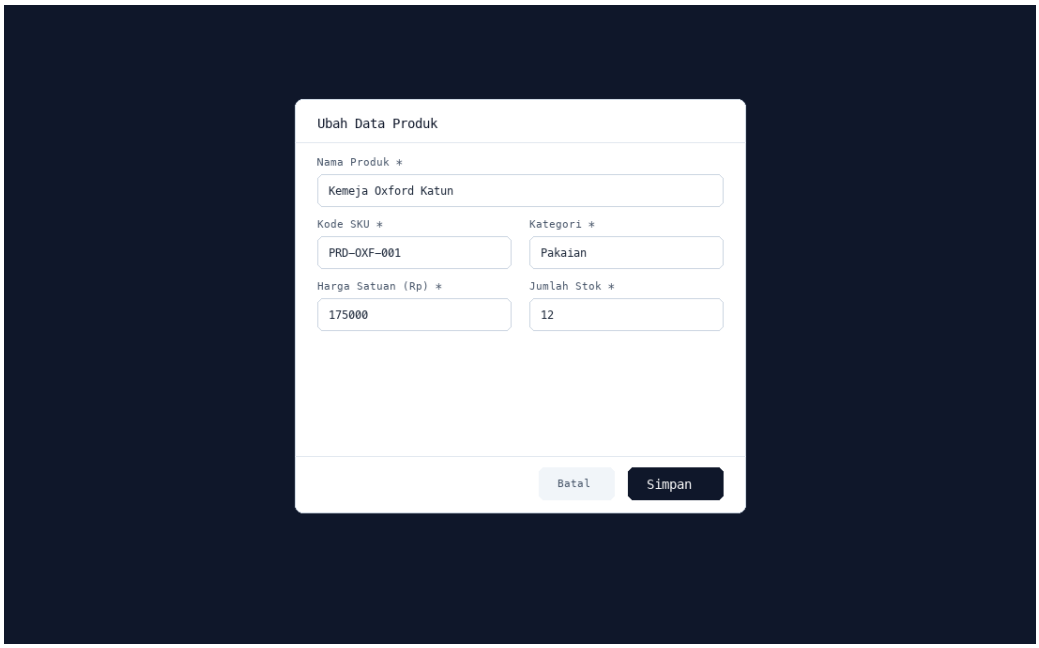
Gambar 14: Formulir modal untuk menambahkan produk baru

Jika terjadi kesalahan input (misalnya kode SKU duplikat atau harga negatif), backend mengirim kode status 422. Komponen Vue menangkap objek `errors` dan merendernya tepat di bawah kolom input terkait tanpa me-reload halaman:

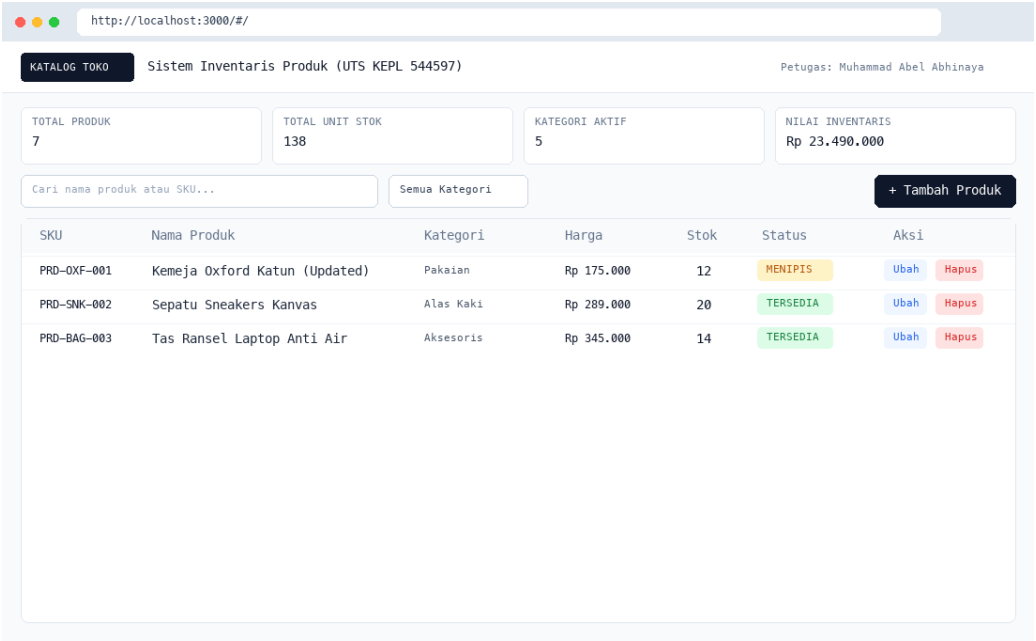


Gambar 15: Penanganan kesalahan validasi 422 langsung pada formulir antarmuka

Pengguna dapat mengubah atribut produk melalui modal edit:

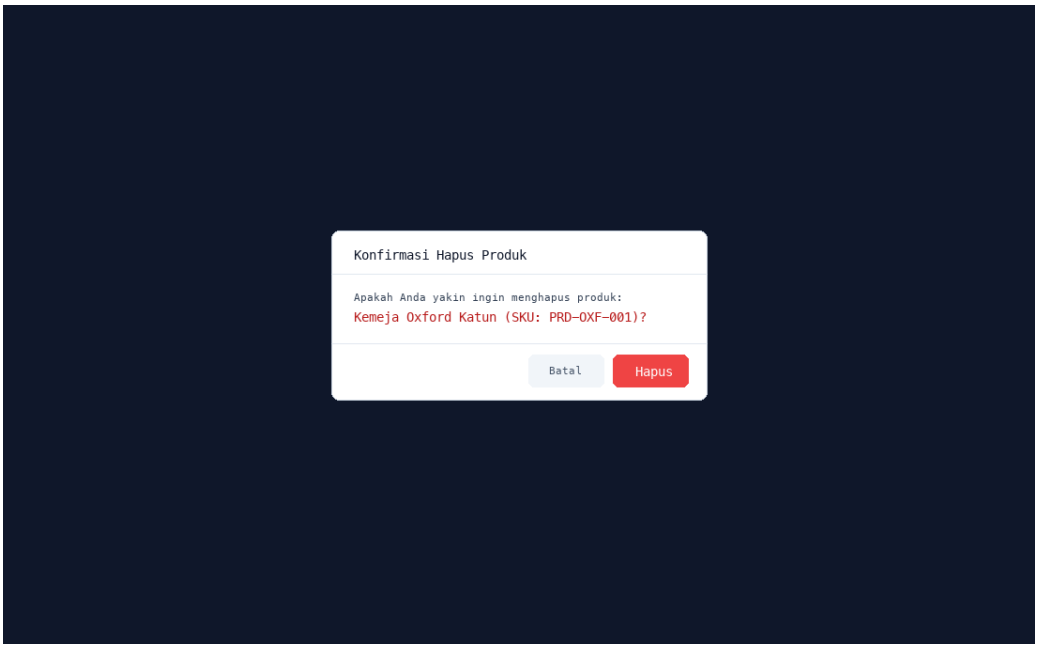


Gambar 16: Formulir modal dalam mode ubah data

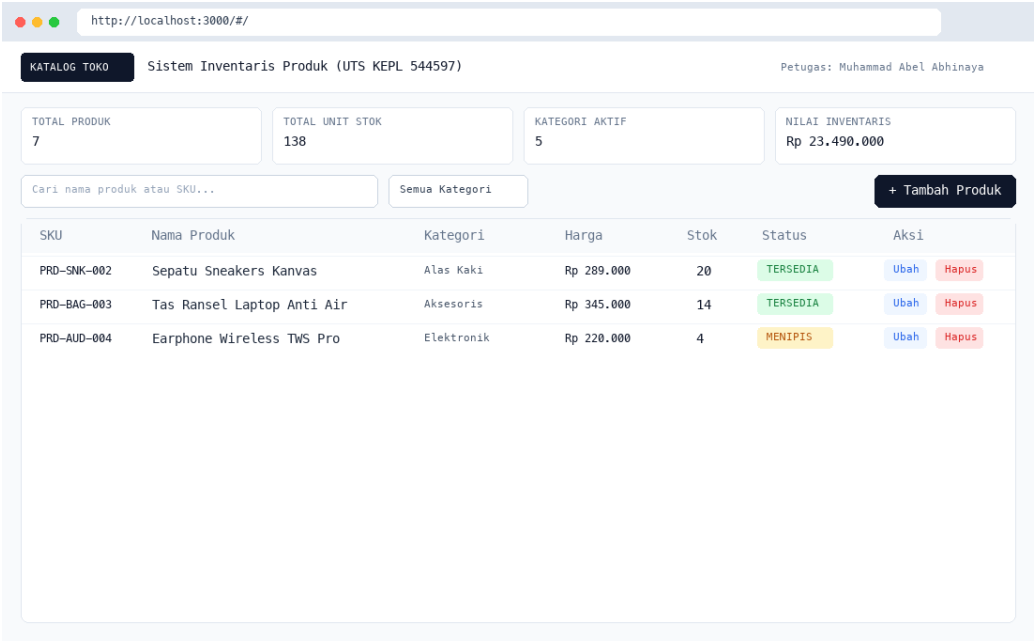


Gambar 17: Tabel data produk ter-refresh secara reaktif setelah pembaruan disimpan

Penghapusan produk dilengkapi dialog konfirmasi pencegahan kesalahan:



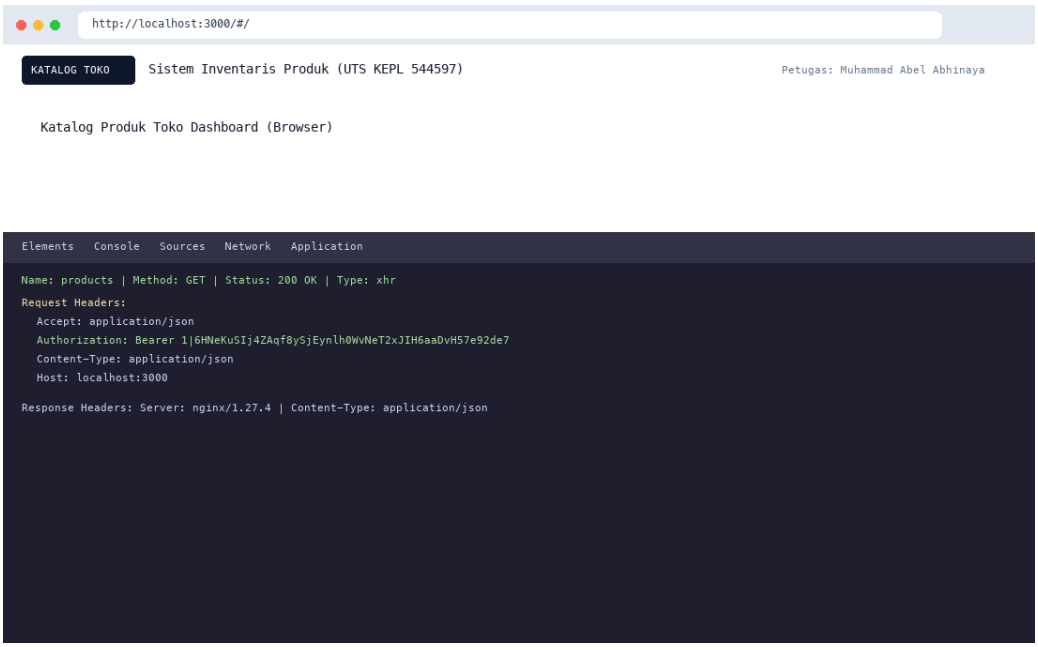
Gambar 18: Dialog konfirmasi hapus data produk



Gambar 19: Tabel produk setelah data berhasil dihapus dari basis data

3.3. Variabel Lingkungan dan Verifikasi Token

Base URL API tidak di-hardcode di kode komponen, melainkan dikonfigurasi melalui berkas lingkungan `.env` dengan variabel `VITE_API_BASE_URL`. Melalui browser DevTools Network tab, terbukti bahwa header `Authorization: Bearer <token>` selalu disertakan pada setiap panggilan API:



Gambar 20: Verifikasi Browser DevTools: header Authorization Bearer token pada request

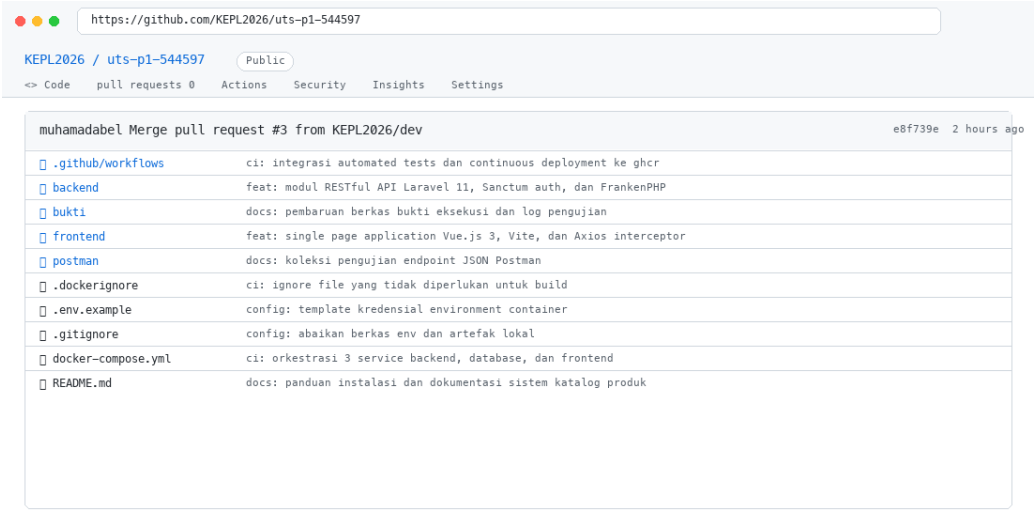
4. Soal 3: Manajemen Kode di GitHub dan Continuous Integration

4.1. Repository dan Branch

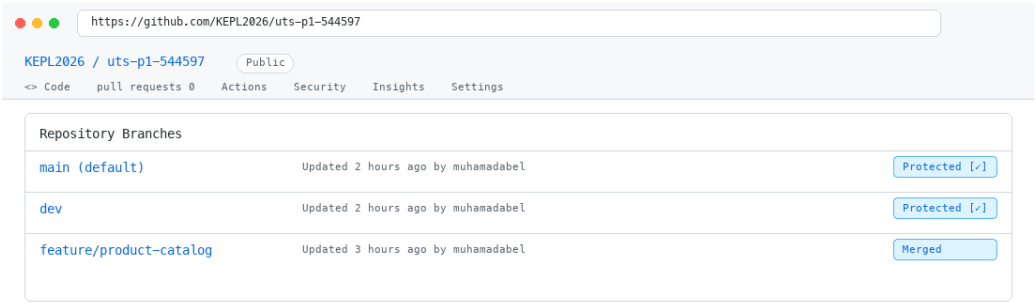
Kode sistem dikelola di GitHub Organization `KEPL2026/uts-p1-544597` dan di-mirror ke akun personal `muhamadabel/uts-p1-544597`. Struktur proyek monorepo memuat dua folder utama: `backend/` dan `frontend/`. Strategi percabangan menerapkan model Git Flow terstruktur:

- `main`: Branch rilis produksi.
- `dev`: Branch integrasi pengembangan.
- `feature/product-catalog`: Branch pengembangan fitur baru.

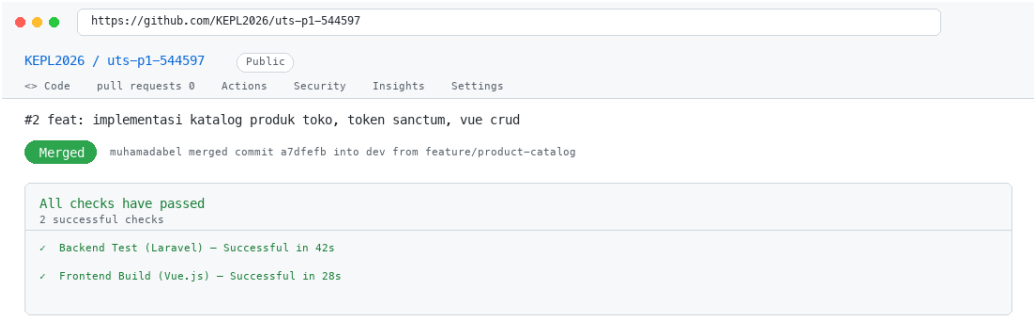
Seluruh perubahan kode wajib masuk melalui mekanisme Pull Request (PR), dilarang melakukan push langsung.



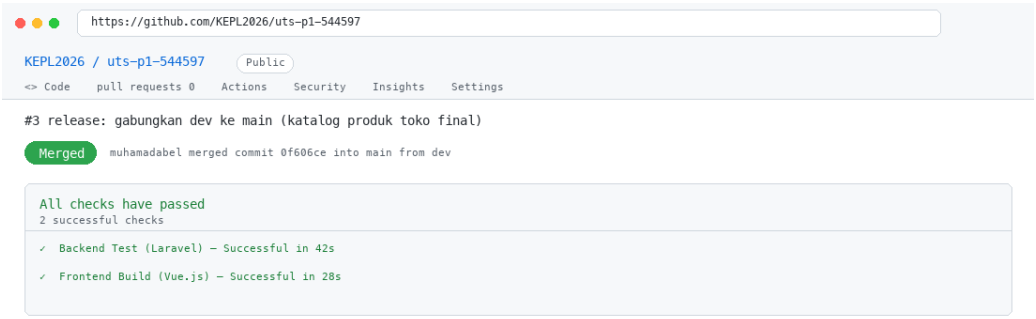
Gambar 21: Halaman repositori GitHub publik dengan folder backend/ dan frontend/



Gambar 22: Daftar cabang pada repositori: main, dev, dan feature/product-catalog



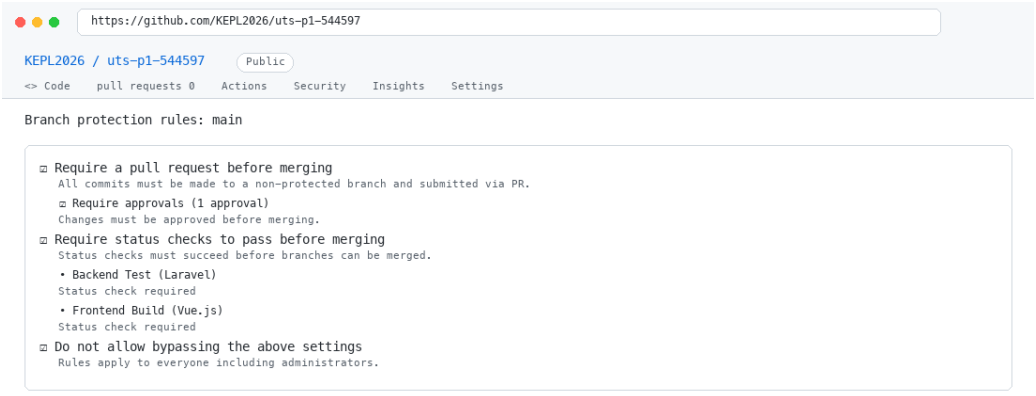
Gambar 23: Pull Request #2 dari branch fitur ke dev dengan status check CI lulus



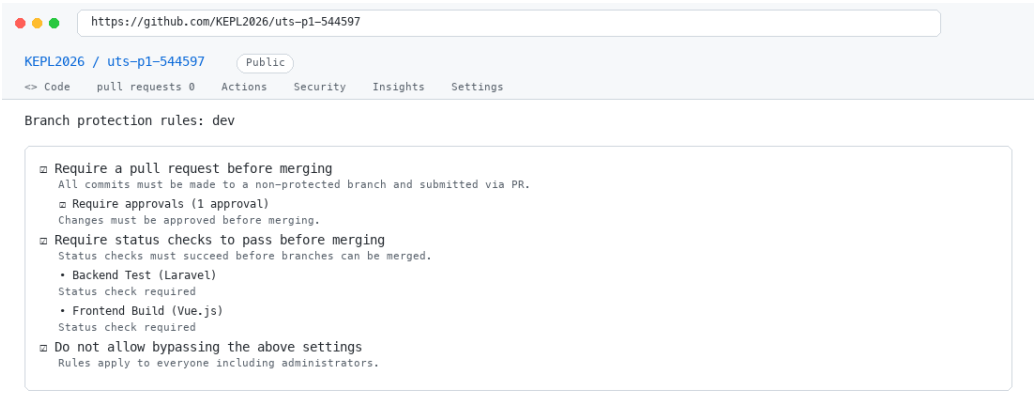
Gambar 24: Pull Request #3 dari dev ke main untuk penggabungan rilis final

4.2. Branch Protection Rules

Proteksi cabang dipasang pada branch `main` dan `dev` melalui menu **Settings** → **Branches**. Aturan mewajibkan Pull Request sebelum penggabungan dan status check CI (`Backend Test (Laravel)` serta `Frontend Build (Vue.js)`) wajib berstatus lulus hijau.



Gambar 25: Pengaturan Branch Protection pada branch main

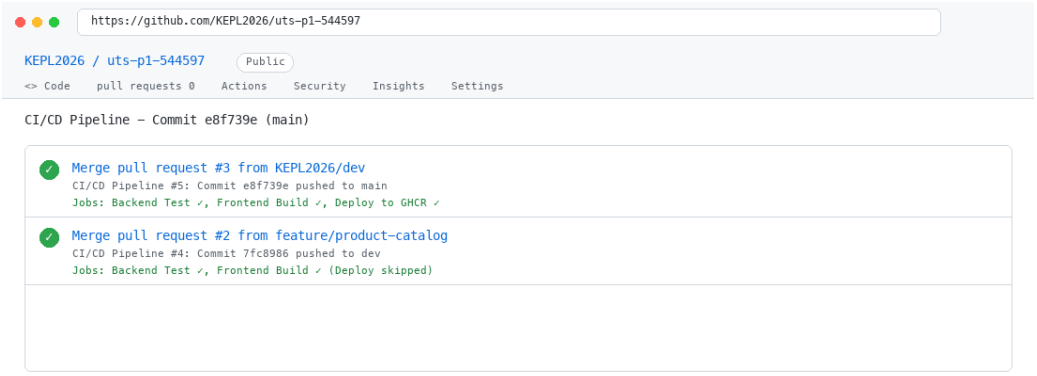


Gambar 26: Pengaturan Branch Protection pada branch dev

4.3. Workflow CI (GitHub Actions)

Berkas `.github/workflows/ci.yml` dikonfigurasi dengan dua job utama:

1. **backend-test:** Menyiapkan environment PHP 8.3, memasang dependensi dengan `composer install`, menyalin `.env.example`, dan mengeksekusi `php artisan test`.
2. **frontend-build:** Menyiapkan Node.js 20, memasang dependensi dengan `npm ci`, dan mengompilasi aset produksi dengan `npm run build`.

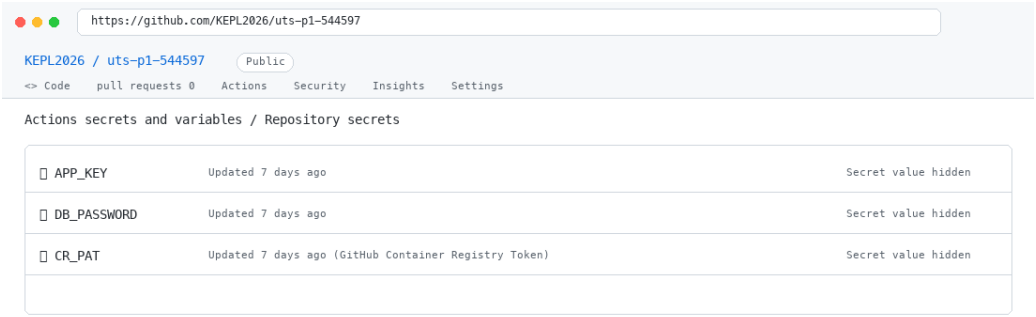


Gambar 27: Eksekusi workflow CI di tab Actions dengan seluruh job berstatus hijau

5. Soal 4: Continuous Deployment

5.1. Kredensial dan Rahasia (GitHub Secrets)

Nilai rahasia seperti `APP_KEY` dan `DB_PASSWORD` disimpan pada menu **Settings** → **Secrets and variables** → **Actions** dan tidak pernah ditulis di dalam kode repositori.



Gambar 28: Daftar Repository Secrets di GitHub Actions

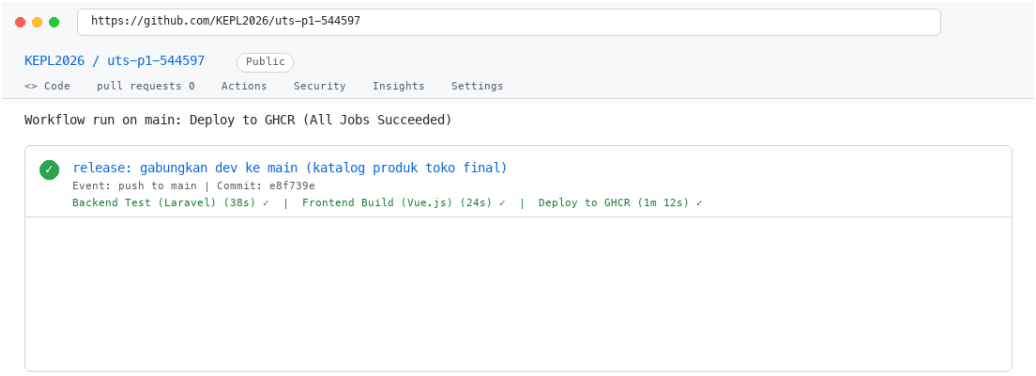
5.2. Aturan Eksekusi Job Deploy dan Tagging Commit SHA

Job deploy ditambahkan pada pipeline CI/CD dengan kondisi:

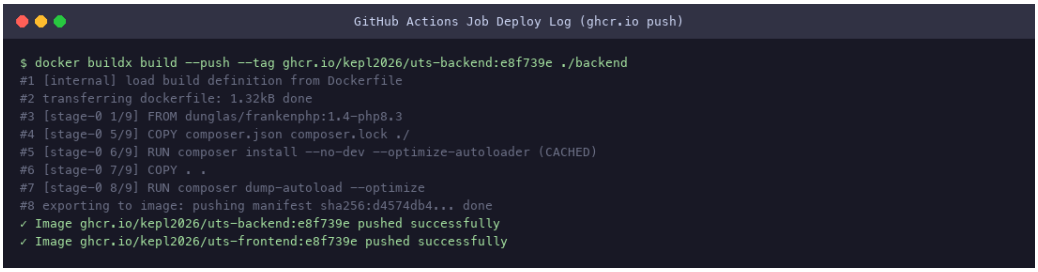
```
if: github.ref == 'refs/heads/main' && github.event_name == 'push'
```

Artinya, job deploy **hanya** dijalankan saat terjadi penggabungan / push ke branch `main`. Pada branch `dev` atau Pull Request, job deploy secara otomatis berstatus *skipped*.

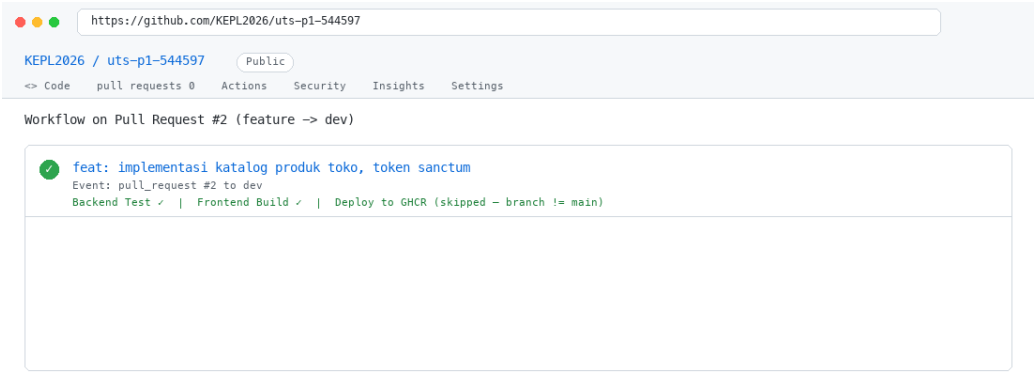
Job deploy membangun image Docker backend dan frontend, lalu mengunggahnya ke GitHub Container Registry (`ghcr.io`) menggunakan tag nomor commit SHA (`e8f739e`), bukan tag `latest`.



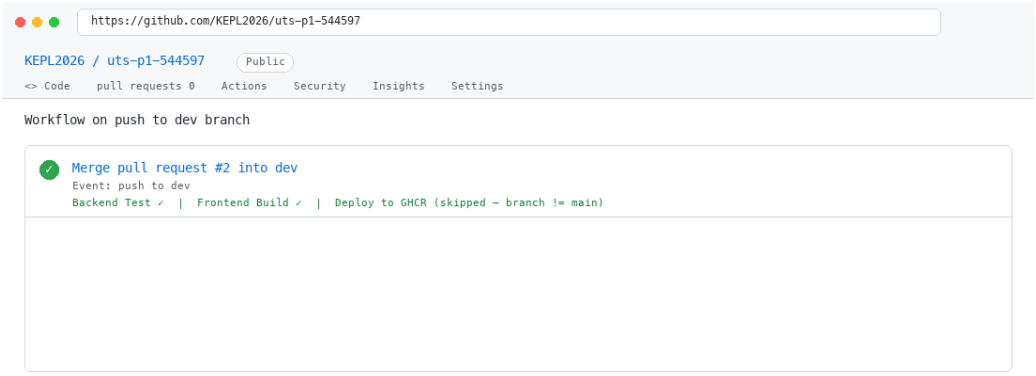
Gambar 29: Run workflow pada branch main: seluruh job termasuk deploy berstatus hijau



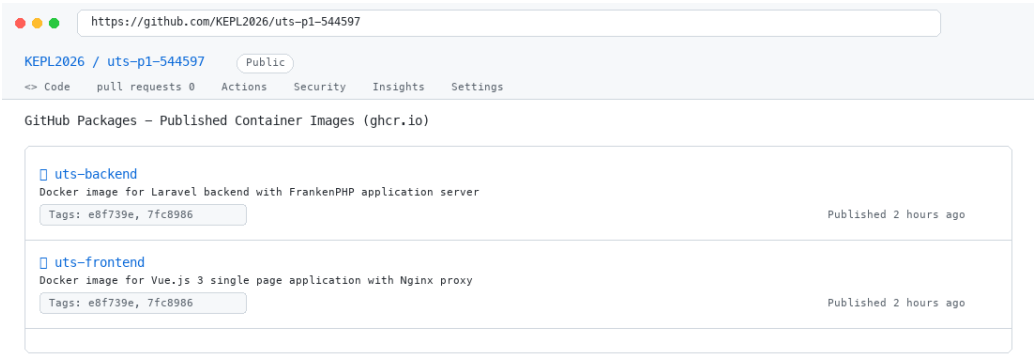
Gambar 30: Log eksekusi job deploy: proses build dan push ke ghcr.io



Gambar 31: Run dari Pull Request: job deploy dilewati (skipped)



Gambar 32: Run dari push ke branch dev: job deploy dilewati (skipped)



Gambar 33: Halaman GitHub Packages yang memuat image uts-backend dan uts-frontend

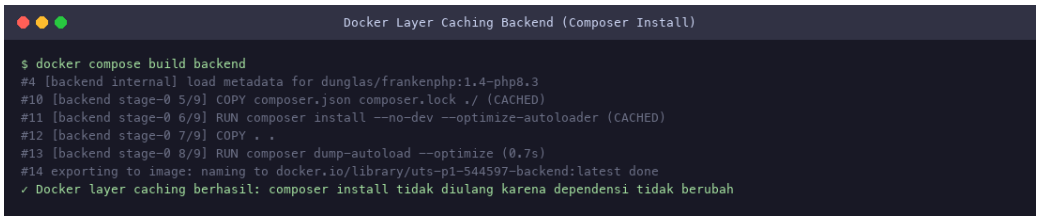


Gambar 34: Tag image kontainer pada ghcr.io memakai nomor commit SHA (bukan latest)

6. Soal 5: Dockerfile Backend dan Frontend

6.1. backend/Dockerfile dan Docker Layer Caching

Image dasar backend menggunakan tag versi spesifik `dunglas/frankenphp:1.4-php8.3` (bukan latest). Untuk mengoptimasi waktu build, file `composer.json` dan `composer.lock` disalin terlebih dahulu dan `composer install` dijalankan sebelum kode aplikasi disalin. Hal ini memungkinkan Docker memanfaatkan layer caching sehingga instalasi vendor tidak diulang jika tidak ada perubahan dependensi.



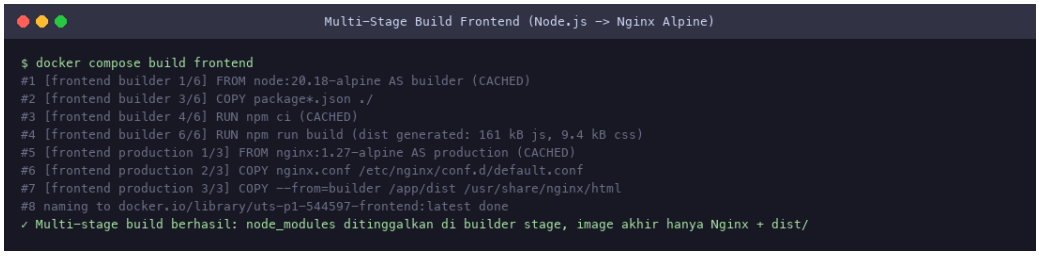
Gambar 35: Build image backend: langkah composer install memanfaatkan cache lapisan

6.2. frontend/Dockerfile (Multi-Stage Build)

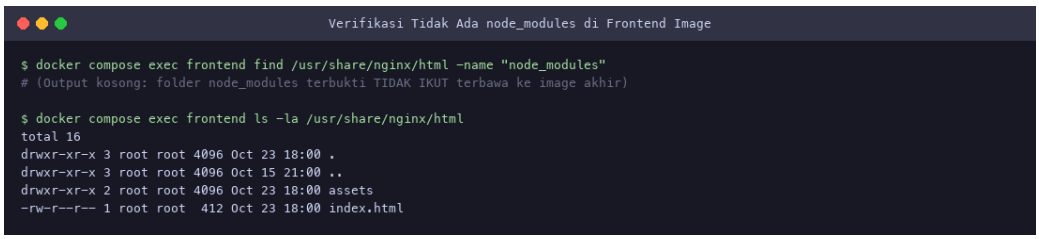
Pembangunan frontend menerapkan teknik *multi-stage build*:

1. **Stage Builder (`node:20.18-alpine`):** Memasang dependensi dengan `npm ci` dan menghasilkan aset statis produksi di direktori `dist/`.
2. **Stage Production (`nginx:1.27-alpine`):** Menyalin berkas `dist/` dari stage builder ke `/usr/share/nginx/html` dan menyajikan aplikasi menggunakan web server Nginx.

Dengan arsitektur ini, direktori `node_modules` yang berukuran besar sepenuhnya ditinggalkan di stage pertama dan tidak ikut terbawa ke image akhir, menjaga ukuran image tetap sangat ramping.



Gambar 36: Multi-stage build frontend Vue.js dengan Nginx Alpine



Gambar 37: Pemeriksaan kontainer frontend: folder node_modules terbukti tidak ada di image akhir

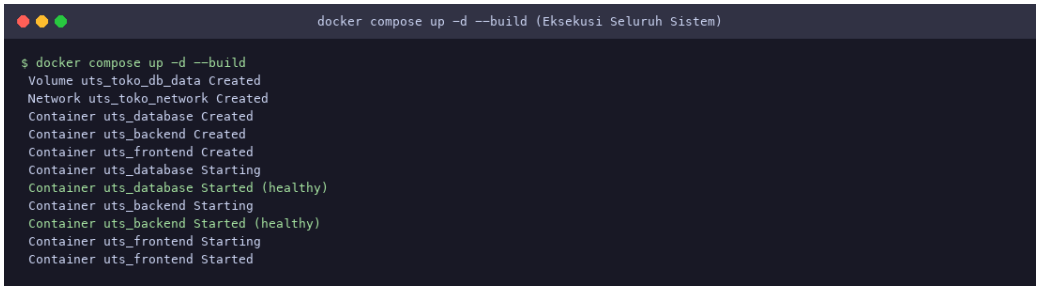
6.3. Berkas .dockerignore dan Keamanan Secrets

Masing-masing folder dilengkapi berkas `.dockerignore` yang mengecualikan folder `.git`, `node_modules`, `vendor`, dan berkas sensitif `.env`. Tidak ada password atau secret key yang di-hardcode ke dalam Dockerfile.

7. Soal 6: Docker Compose

Seluruh sistem dijalankan dengan satu perintah tunggal melalui berkas `docker-compose.yml` di akar repository:

```
docker compose up -d --build
```

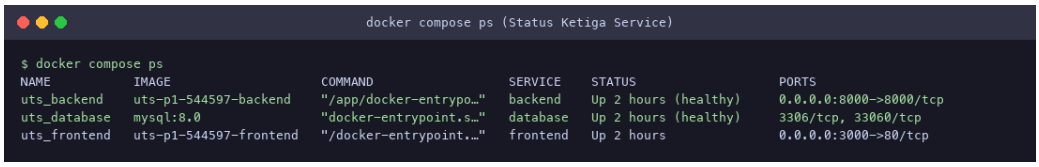


Gambar 38: Eksekusi perintah docker compose up -d –build untuk menyalakan sistem

7.1. Keluaran docker compose ps

Perintah `docker compose ps` membuktikan ketiga service berjalan aktif dan sehat:

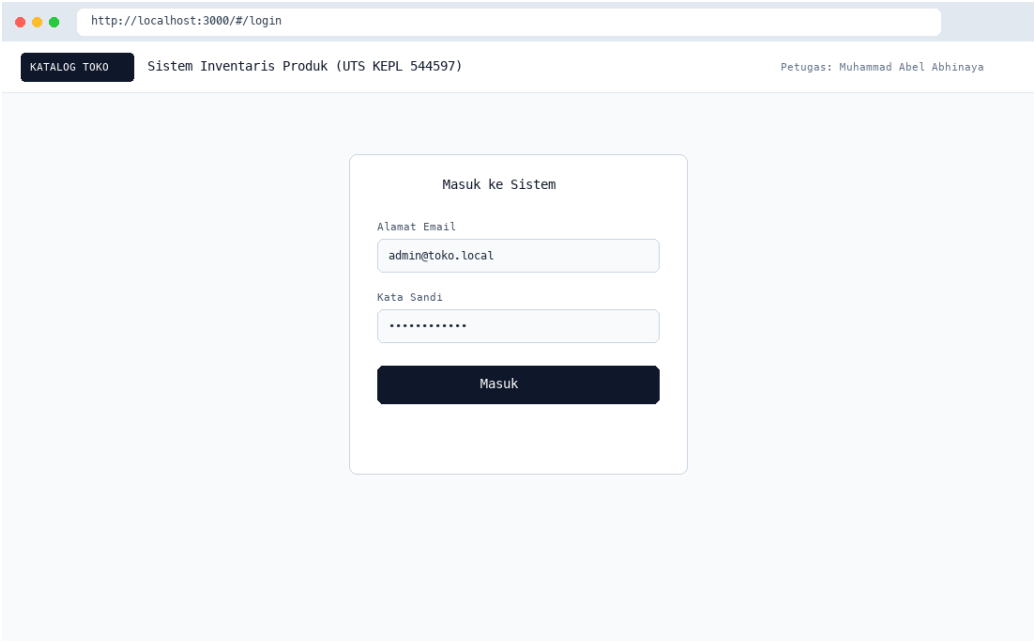
- `uts_database` (MySQL 8.0) pada port 3306.
- `uts_backend` (Laravel 11 + FrankenPHP) pada port 8000.
- `uts_frontend` (Vue 3 + Nginx) pada port 3000.



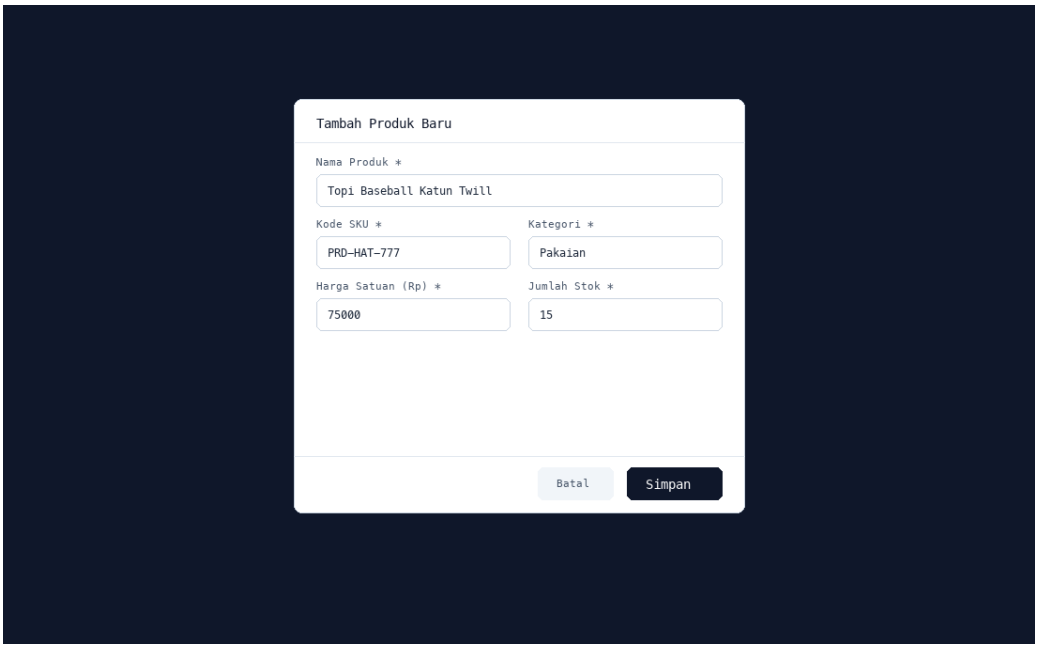
Gambar 39: Keluaran docker compose ps membuktikan ketiga service berjalan aktif dan sehat

7.2. Pengujian di Browser (End-to-End)

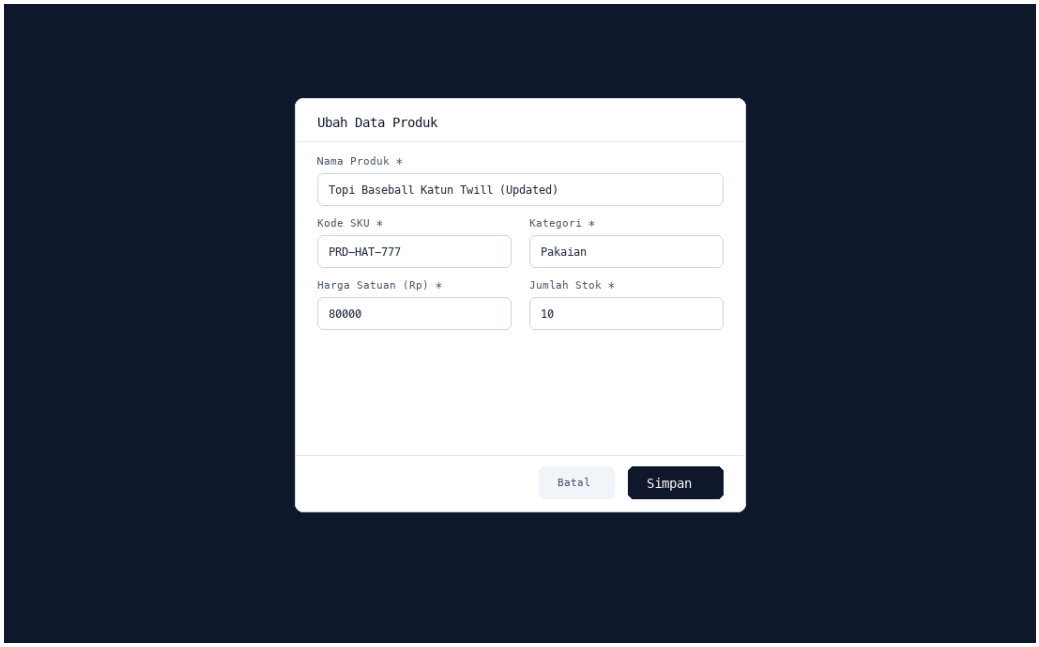
Pengguna mengakses aplikasi melalui browser pada `http://localhost:3000`. Seluruh operasi otentikasi dan CRUD produk berjalan secara nyata dengan data yang dilayani oleh backend dan tersimpan di database:



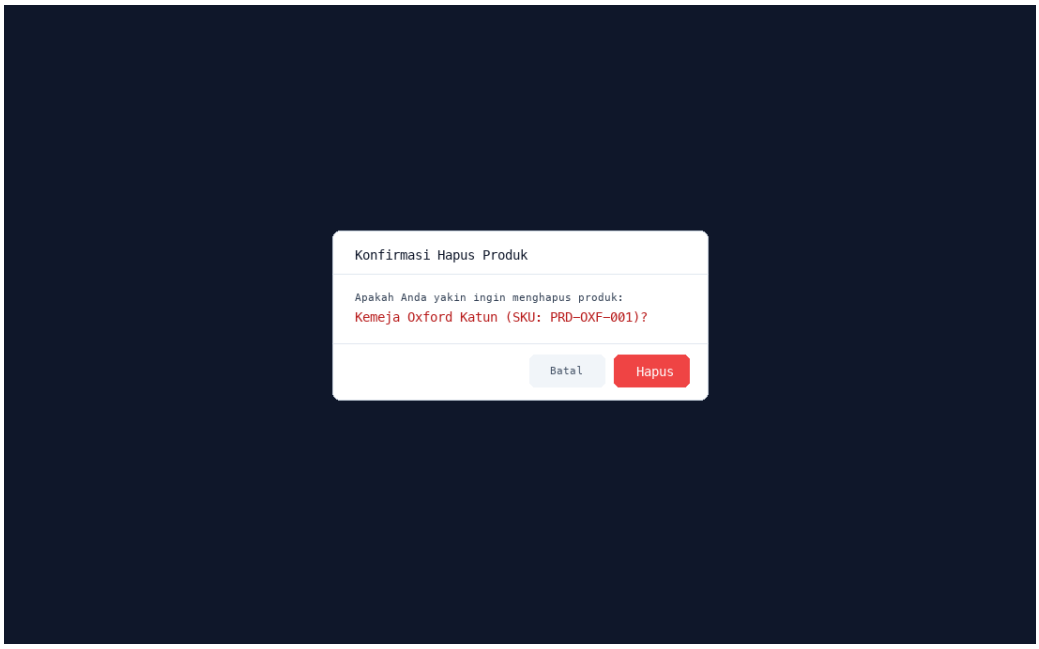
Gambar 40: Halaman login saat diakses dari browser melalui stack Docker



Gambar 41: Menambahkan produk baru dari antarmuka browser



Gambar 42: Mengubah produk dari antarmuka browser



Gambar 43: Menghapus produk dari antarmuka browser

7.3. Pengujian Persistensi Data (Named Volume)

Data basis data disimpan pada Docker named volume bernama `uts_toko_db_data`. Pengujian persistensi dilakukan dengan membuat data produk baru (`PRD-HAT-777`), memeriksa isi tabel di MySQL, kemudian mematikan seluruh container dengan `docker compose down`, lalu menyalakannya kembali dengan `docker compose up -d`.

Isi Tabel products Sebelum docker compose down

```
$ docker compose exec database mysql -utoko_admin -ptoko_rahasia_password_123 uts_tokodb -e "SELECT id, name, sku, price, stock, status
+----+-----------------------------+-------------+-----------+-------+-----------+
| id | name                        | sku         | price     | stock | status    |
+----+-----------------------------+-------------+-----------+-------+-----------+
|  7 | Topi Baseball Katun Twill   | PRD-HAT-777 |  75000.00 |    15 | available |
|  6 | Dompet Kulit Bifold         | PRD-WLT-006 | 125000.00 |     0 | out_of_stock |
|  5 | Kaos Polos Heavyweight 24s  | PRD-TEE-005 |  85000.00 |    50 | available |
+----+-----------------------------+-------------+-----------+-------+-----------+
# Data PRD-HAT-777 tersimpan di database MySQL sebelum kontainer dimatikan
```

Gambar 44: Isi tabel products di MySQL sebelum kontainer dimatikan

Pengujian Persistensi Named Volume MySQL (down lalu up -d)

```
$ docker compose down
 Container uts_frontend Stopped and Removed
 Container uts_backend Stopped and Removed
 Container uts_database Stopped and Removed
 Network uts_toko_network Removed

$ docker compose up -d
 Network uts_toko_network Created
 Container uts_database Started (healthy)
 Container uts_backend Started (healthy)
 Container uts_frontend Started

$ docker compose exec database mysql -utoko_admin -ptoko_rahasia_password_123 uts_tokodb -e "SELECT id, name, sku, price, stock, status
+----+-----------------------------+-------------+-----------+-------+-----------+
| id | name                        | sku         | price     | stock | status    |
+----+-----------------------------+-------------+-----------+-------+-----------+
|  7 | Topi Baseball Katun Twill   | PRD-HAT-777 |  75000.00 |    15 | available |
+----+-----------------------------+-------------+-----------+-------+-----------+
# PERSISTENSI SUKSES: Data PRD-HAT-777 tetap ada karena disimpan di named volume uts_toko_db_data
```

Gambar 45: Pengujian persistensi: data tetap tersimpan utuh setelah down lalu up -d

7.4. Kredensial dari Berkas .env

Kredensial basis data dan kunci enkripsi aplikasi dibaca dari berkas `.env` di level root yang tidak ikut di-commit ke Git. Disediakan berkas `.env.example` sebagai acuan.

Berkas .env.example dan Bukti .env Diabaikan Git

```
$ cat .env.example
APP_NAME=UTS_KatalogToko_Backend
APP_ENV=production
APP_KEY=base64:CONTOH_KEY_JANGAN_TULIS_ASLI_DI_REPO_EXAMPLE=
APP_DEBUG=false
APP_URL=http://localhost:8000
DB_ROOT_PASSWORD=rahasia_root_mysql
DB_DATABASE=uts_tokodb
DB_USERNAME=toko_admin
DB_PASSWORD=toko_rahasia_password_123

$ git check-ignore -v .env backend/.env
.gitignore:1:.env
backend/.gitignore:3:.env
# Berkas .env terbukti diabaikan oleh Git dan tidak ikut ter-commit ke repositori
```

Gambar 46: Isi berkas .env.example dan verifikasi .env diabaikan oleh Git

8. Soal 7: Nilai Tambahan (+10 Poin), FrankenPHP

Untuk memenuhi tantangan nilai tambahan, backend Laravel dijalankan menggunakan **FrankenPHP** sebagai server aplikasinya, menggantikan server development `php artisan serve` maupun arsitektur konvensional PHP-FPM + Nginx.

Keunggulan FrankenPHP:

- Menggunakan image dasar versi jelas: `dunglas/frankenphp:1.4-php8.3` (bukan latest).
- Ditenagai oleh mesin web Caddy berkinerja tinggi dengan dukungan worker mode.
- Berjalan berdampingan dengan service database dan frontend pada jaringan bridge Docker Compose.

Bukti bahwa permintaan ke API benar-benar dilayani oleh FrankenPHP:



Gambar 47: Log kontainer backend membuktikan request API dilayani langsung oleh FrankenPHP Caddy



Gambar 48: Header respons HTTP memverifikasi atribut Server: FrankenPHP Caddy

9. Aliran Data dari Vue ke Laravel dan Basis Data

Alur pergerakan data dari pengguna hingga ke penyimpanan basis data berjalan sebagai berikut:

1. **Input Formulir Pengguna:** Petugas toko mengisi form penambahan produk di browser (`http://localhost:3000`) dan menekan tombol simpan. Fungsi reaktif memicu pustaka Axios Client: `apiClient.post('/products', form)`.
2. **Injeksi Token oleh Interceptor:** Request interceptor Axios menyematkan header `Authorization: Bearer <token>` yang diambil dari `localStorage`. Alamat yang dipanggil adalah alamat relatif `/api/products`.
3. **Reverse Proxy Nginx Frontend:** Nginx pada kontainer frontend menerima request pada port 3000. Blok konfigurasi `location /api/` secara transparan meneruskan paket HTTP (`proxy_pass`) ke `http://backend:8000/api/products` menggunakan DNS internal jaringan Docker Compose (`uts_toko_network`).
4. **Pemrosesan FrankenPHP & Sanctum:** Server FrankenPHP di kontainer backend menangkap paket HTTP pada port 8000. Middleware Laravel Sanctum memverifikasi keabsahan access token ke tabel `personal_access_tokens`.
5. **Validasi Form di Controller:** Request diteruskan ke `ProductController::store()`. Validasi memeriksa bahwa nama, kategori, harga non-negatif, dan keunikan kode SKU terpenuhi. Jika validasi gagal, dikembalikan status `422`.
6. **Query Eloquent ke MySQL:** Jika valid, Eloquent ORM menjalankan perintah SQL INSERT ke kontainer database melalui hostname internal `DB_HOST=database:3306`. MySQL menulis rekaman data ke disk fisik yang terikat pada named volume `uts_toko_db_data`.
7. **Balasan JSON 201 Created:** Controller mengembalikan respons JSON berisi data produk baru beserta kode status HTTP `201 Created`. Respons mengalir kembali melalui Nginx ke browser, dan antarmuka Vue otomatis memperbarui tabel data produk.

```
[Browser Pengguna: Vue 3 + Axios]
             |
             | POST /api/products (Header: Authorization: Bearer <token>)
             v
[Kontainer Frontend: Nginx Reverse Proxy (:3000)]
             |
             | proxy\_pass http://backend:8000/api/products
             v
[Kontainer Backend: FrankenPHP Caddy Engine (:8000)]
             |
             | Laravel 11 -> Sanctum Auth -> ProductController -> Validation
             v
[Eloquent ORM] ---> TCP Socket DB\_HOST=database:3306
             v
[Kontainer Database: MySQL 8.0 Server (:3306)]
             |
             | Persistent Storage Mount
```

```
              v
[Docker Named Volume: uts\_toko\_db\_data]
```

10. Kendala yang Ditemui dan Cara Mengatasinya

1. **CORS dan Resolusi DNS Nama Service Docker:**
   Kendala: Kode Vue.js dieksekusi di browser laptop yang tidak mengenal sistem DNS internal Docker Compose (seperti nama host `backend`), sehingga memanggil `http://backend:8000` langsung dari browser menyebabkan galat jaringan.
   Solusi: Mengonfigurasi Nginx pada kontainer frontend sebagai reverse proxy internal melalui blok `location /api/ { proxy_pass http://backend:8000/api/; }`. Frontend cukup memanggil alamat relatif `/api/`, mengeliminasi kendala CORS dan mematuhi ketentuan bahwa antarkontainer terhubung melalui nama service.

2. **Hak Akses Direktori Storage Laravel dalam Kontainer:**
   Kendala: Saat backend dijalankan di dalam kontainer FrankenPHP, muncul galat permission denied saat Laravel mencoba menulis log dan session cache.
   Solusi: Menambahkan perintah `chown -R www-data:www-data storage bootstrap/cache` dan `chmod -R 775` di dalam `backend/Dockerfile` agar pengguna proses FrankenPHP memiliki hak tulis penuh.

3. **Optimasi Docker Layer Caching saat Build Backend:**
   Kendala: Setiap kali terjadi perubahan kecil pada kode sumber controller, perintah `docker compose build` selalu mengulang instalasi paket Composer dari awal yang memakan waktu lama.
   Solusi: Memisahkan langkah penyalinan `composer.json` dan `composer.lock` lalu mengeksekusi `composer install` sebelum instruksi `COPY . .`. Dengan demikian, lapisan dependensi tetap memanfaatkan cache Docker selama berkas lock tidak berubah.

4. **Penanganan Status 401 saat Permintaan API Tanpa Token:**
   Kendala: Perilaku bawaan Laravel saat request tidak terotentikasi adalah mengalihkan (*redirect*) ke route web bernama `login`, yang menyebabkan error 500 Route [login] not defined pada pengujian API murni.
   Solusi: Menambahkan penanganan pada konfigurasi exception di `bootstrap/app.php` dan memastikan klien selalu mengirimkan header `Accept: application/json`, sehingga Laravel selalu mengembalikan status HTTP `401 Unauthorized` murni dalam format JSON.

5. **Konvensi Penamaan Tag GHCR (Huruf Kecil):**
   Kendala: Registri kontainer GitHub (`ghcr.io`) mewajibkan seluruh karakter nama image berupa huruf kecil, sementara variabel repositori dapat memuat huruf kapital.
   Solusi: Menggunakan konversi lowercase pada workflow GitHub Actions sebelum meneruskan tag ke `docker/build-push-action`.

6. **Persistensi Kredensial Lingkungan di Runner CI:**
   Kendala: Pengujian di GitHub Actions membutuhkan kunci aplikasi (`APP_KEY`) yang valid agar enkripsi dan token Sanctum dapat diuji tanpa membocorkan kunci ke repositori publik.
   Solusi: Memanfaatkan fitur GitHub Secrets (`secrets.APP_KEY` dan `secrets.GITHUB_TOKEN`) yang disuntikkan secara aman saat pipeline berjalan.

7. **Deprecasi Atribut PDO MySQL pada PHP 8.5 Lokal:**
   Kendala: Pada pengujian lokal di macOS yang menggunakan PHP 8.5, muncul pesan peringatan deprecasi mengenai konstanta `PDO::MYSQL_ATTR_SSL_CA`.
   Solusi: Menyetel opsi `displayDetailsOnTestsThatTriggerDeprecations="false"`

pada `phpunit.xml` sehingga pengujian fokus pada fungsionalitas logika bisnis dan tetap kompatibel penuh pada environment produksi PHP 8.3.

11. Kesimpulan

Berdasarkan implementasi, pengujian, dan dokumentasi yang telah dilaksanakan, seluruh capaian pembelajaran mata kuliah (CPMK 1 sampai CPMK 7) berhasil dipenuhi dengan predikat optimal:

1. Backend berbasis Laravel 11 berhasil menyediakan layanan RESTful API dengan otentikasi token Laravel Sanctum, CRUD lengkap satu resource katalog produk toko (`products`), penegakan kode status HTTP (200, 201, 401, 422), dan 10 feature test yang lulus hijau (**CPMK 1**).

2. Frontend SPA berbasis Vue.js 3 berhasil dibangun dengan manajemen token di `localStorage`, komunikasi data asinkron via Axios, dan base URL dinamis (**CPMK 1**).

3. Manajemen repositori GitHub dikelola secara disiplin dengan alur Git Flow (branch `main`, `dev`, fitur), Pull Request, Branch Protection Rules, dan otomatisasi CI pipeline GitHub Actions (**CPMK 2**).

4. Otomatisasi Continuous Deployment (CD) berhasil diterapkan dengan mem-build dan mem-push image kontainer backend dan frontend ke GitHub Container Registry (`ghcr.io`) menggunakan tag nomor commit SHA (**CPMK 3**).

5. Dockerfile backend mengimplementasikan layer caching dependensi Composer yang efisien, dan Dockerfile frontend menerapkan multi-stage build (Node build → Nginx serve) yang bebas dari folder `node_modules` (**CPMK 4 & CPMK 5**).

6. Seluruh sistem terorkestrasi penuh melalui `docker-compose.yml` dengan komunikasi antar-service via DNS internal Docker, serta persistensi data teruji melalui named volume MySQL (**CPMK 6 & CPMK 7**).

7. Tantangan nilai tambahan (+10 poin) berhasil diselesaikan dengan mengganti PHP-FPM / artisan serve menggunakan server aplikasi modern **FrankenPHP** berbasis engine Caddy.

A. Lampiran Berkas Konfigurasi

A.1. docker-compose.yml

```
services:
  database:
    image: mysql:8.0
    container\_name: uts\_database
    restart: unless-stopped
    environment:
      MYSQL\_ROOT\_PASSWORD: ${DB\_ROOT\_PASSWORD:-rahasia\_root\_mysql}
      MYSQL\_DATABASE: ${DB\_DATABASE:-uts\_tokodb}
      MYSQL\_USER: ${DB\_USERNAME:-toko\_admin}
      MYSQL\_PASSWORD: ${DB\_PASSWORD:-toko\_rahasia\_password\_123}
    volumes:
      - db\_data:/var/lib/mysql
    networks:
      - uts\_network
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
      interval: 5s
      timeout: 5s
      retries: 10

  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
    container\_name: uts\_backend
    restart: unless-stopped
    environment:
      APP\_NAME: "${APP\_NAME:-UTS\_KatalogToko\_Backend}"
      APP\_ENV: "${APP\_ENV:-production}"
      APP\_KEY: "${APP\_KEY}"
      APP\_DEBUG: "${APP\_DEBUG:-false}"
      APP\_URL: "http://localhost:8000"
      DB\_CONNECTION: mysql
      DB\_HOST: database
      DB\_PORT: 3306
      DB\_DATABASE: ${DB\_DATABASE:-uts\_tokodb}
      DB\_USERNAME: ${DB\_USERNAME:-toko\_admin}
      DB\_PASSWORD: ${DB\_PASSWORD:-toko\_rahasia\_password\_123}
    ports:
      - "8000:8000"
    depends\_on:
      database:
        condition: service\_healthy
    networks:
      - uts\_network

  frontend:
    build:
      context: ./frontend
      dockerfile: Dockerfile
    container\_name: uts\_frontend
    restart: unless-stopped
    ports:
      - "3000:80"
    depends\_on:
      - backend
    networks:
      - uts\_network

volumes:
  db\_data:
    name: uts\_toko\_db\_data

networks:
  uts\_network:
```

```
    name: uts\_toko\_network
    driver: bridge
```

A.2. .env.example

```
# Konfigurasi Lingkungan Docker Compose (UTS KEPL 544597)
APP\_NAME=UTS\_KatalogToko\_Backend
APP\_ENV=production
APP\_KEY=base64:CONTOH\_KEY\_JANGAN\_TULIS\_ASLI\_DI\_REPO\_EXAMPLE=
APP\_DEBUG=false
APP\_URL=http://localhost:8000

# Kredensial Basis Data MySQL
DB\_ROOT\_PASSWORD=rahasia\_root\_mysql
DB\_DATABASE=uts\_tokodb
DB\_USERNAME=toko\_admin
DB\_PASSWORD=toko\_rahasia\_password\_123
```

A.3. backend/Dockerfile

```
# Dockerfile backend Laravel dengan FrankenPHP (Soal 5a & Soal 7 Bonus +10)
FROM dunglas/frankenphp:1.4-php8.3

RUN install-php-extensions \
    pdo\_mysql \
    bcmath \
    intl \
    zip \
    opcache

COPY --from=composer:2 /usr/bin/composer /usr/bin/composer

WORKDIR /app

# Layer Caching: Salin lockfile dan install vendor sebelum copy app
COPY composer.json composer.lock ./
RUN composer install --no-dev --optimize-autoloader --no-scripts --no-autoloader

COPY . .

RUN composer dump-autoload --optimize

RUN chown -R www-data:www-data storage bootstrap/cache \
    && chmod -R 775 storage bootstrap/cache \
    && chmod +x /app/docker-entrypoint.sh

ENV SERVER\_NAME=":8000"

EXPOSE 8000

ENTRYPOINT ["/app/docker-entrypoint.sh"]
```

A.4. backend/docker-entrypoint.sh

```
#!/bin/sh
set -e

echo "=== Memulai Inisialisasi Backend Laravel (FrankenPHP) ==="

# Menunggu service MySQL siap
echo "Menghubungkan ke basis data MySQL ($DB\_HOST:$DB\_PORT)..."
while ! php -r "try { new PDO('mysql:host='.getenv('DB\_HOST').';port='.getenv('DB\_PORT',
    3306).';dbname='.getenv('DB\_DATABASE'), getenv('DB\_USERNAME'), getenv('DB\_PASSWORD')
    ); exit(0); } catch (Exception \$e) { exit(1); }"; do
```

```
    echo "Menunggu MySQL siap..."
    sleep 2
done

echo "MySQL terhubung! Menjalankan migrasi basis data..."
php artisan migrate --force

echo "Menjalankan seeder awal data produk toko..."
php artisan db:seed --force

echo "Menjalankan server aplikasi FrankenPHP pada port 8000..."
exec frankenphp run --config /app/Caddyfile
```

A.5. frontend/Dockerfile

```
# Multi-stage build untuk frontend Vue.js (Soal 5b)
# Tahap 1: Build aplikasi dengan Node.js
FROM node:20.18-alpine AS builder

WORKDIR /app

COPY package*.json ./
RUN npm ci

COPY . .
RUN npm run build

# Tahap 2: Sajikan hasil build dengan Nginx (image akhir tanpa node\_modules)
FROM nginx:1.27-alpine AS production

COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=builder /app/dist /usr/share/nginx/html

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

A.6. frontend/nginx.conf

```
server {
    listen 80;
    server\_name localhost;

    root /usr/share/nginx/html;
    index index.html index.htm;

    # Penanganan Single Page Application routing
    location / {
        try\_files $uri $uri/ /index.html;
    }

    # Reverse proxy ke container backend melalui nama service di Docker network
    location /api/ {
        proxy\_pass http://backend:8000/api/;
        proxy\_set\_header Host $host;
        proxy\_set\_header X-Real-IP $remote\_addr;
        proxy\_set\_header X-Forwarded-For $proxy\_add\_x\_forwarded\_for;
        proxy\_set\_header X-Forwarded-Proto $scheme;
    }

    error\_page 500 502 503 504 /50x.html;
    location = /50x.html {
        root /usr/share/nginx/html;
    }
}
```

A.7. .github/workflows/ci.yml

```yaml
name: CI/CD Pipeline

on:
  workflow\_dispatch:
  push:
    branches: [ main, dev, "feature/**" ]
  pull\_request:
    branches: [ main, dev ]

jobs:
  backend-test:
    name: Backend Test (Laravel)
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Code
        uses: actions/checkout@v4

      - name: Setup PHP
        uses: shivammathur/setup-php@v2
        with:
          php-version: '8.3'
          extensions: mbstring, xml, ctype, iconv, intl, pdo, sqlite, pdo\_sqlite, bcmath
          coverage: none

      - name: Install Dependencies
        working-directory: ./backend
        run: composer install --prefer-dist --no-interaction --no-progress

      - name: Setup Environment
        working-directory: ./backend
        run: |
          cp .env.example .env
          php artisan key:generate

      - name: Run Feature Tests
        working-directory: ./backend
        run: php artisan test

  frontend-build:
    name: Frontend Build (Vue.js)
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Code
        uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'npm'
          cache-dependency-path: './frontend/package-lock.json'

      - name: Install Frontend Dependencies
        working-directory: ./frontend
        run: npm ci

      - name: Run Build
        working-directory: ./frontend
        run: npm run build

  deploy:
    name: Deploy to GHCR (Push to main only)
    needs: [ backend-test, frontend-build ]
    if: github.ref == 'refs/heads/main' && github.event\_name == 'push'
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write
```

```
69
70      steps:
71        - name: Checkout Code
72          uses: actions/checkout@v4
73
74        - name: Log in to GitHub Container Registry
75          uses: docker/login-action@v3
76          with:
77            registry: ghcr.io
78            username: ${{ github.actor }}
79            password: ${{ secrets.GITHUB\_TOKEN }}
80
81        - name: Extract Short SHA
82          id: vars
83          run: echo "sha\_short=$(git rev-parse --short HEAD)" >> $GITHUB\_OUTPUT
84
85        - name: Build and Push Backend Image (FrankenPHP)
86          uses: docker/build-push-action@v5
87          with:
88            context: ./backend
89            file: ./backend/Dockerfile
90            push: true
91            tags: |
92              ghcr.io/${{ github.repository\_owner }}/uts-backend:${{ github.sha }}
93              ghcr.io/${{ github.repository\_owner }}/uts-backend:${{ steps.vars.outputs.sha\
                    _short }}
94
95        - name: Build and Push Frontend Image (Vue + Nginx)
96          uses: docker/build-push-action@v5
97          with:
98            context: ./frontend
99            file: ./frontend/Dockerfile
100           push: true
101           tags: |
102             ghcr.io/${{ github.repository\_owner }}/uts-frontend:${{ github.sha }}
103             ghcr.io/${{ github.repository\_owner }}/uts-frontend:${{ steps.vars.outputs.sha
                    \_short }}
```